Standardization of design templates for architectural layouts

DS203 Sem 2 2024
Group 39
Anubha Vyasamudri (210100019)
Mariyam Saqib (210040090)
Riddhi Agarwal (210040124)
Om Nagalkar (21D110011)

Objective

Data available:
1183 views (i.e., buildings)
architected by the company into
bitmaps of size 640 x 480 pixels

1. To group the designs into families, based on their shapes, in order to standardize them and create design templates

2. To recognize and implement criteria based on which to classify desgins based on complexity

3. To facilitate retrieval of relevant layouts based on input specifications

Steps

1. **Data Preprocessing:** conversion to grayscale and isolating the yellow image

2. **PCA:** Dimensionality Reduction

3. **Feature extraction:** Extracting relevant features, like the criteria while grouping based on complexity and resizing and flattening to a 1D array while grouping based on shape

4. **Clustering**: Using k-means and other clustering algorithms to create categories based on extracted features

5. **Visualization**: Visualization of scatter plot and storing the clusters as dictionaries of designs

Solution summary

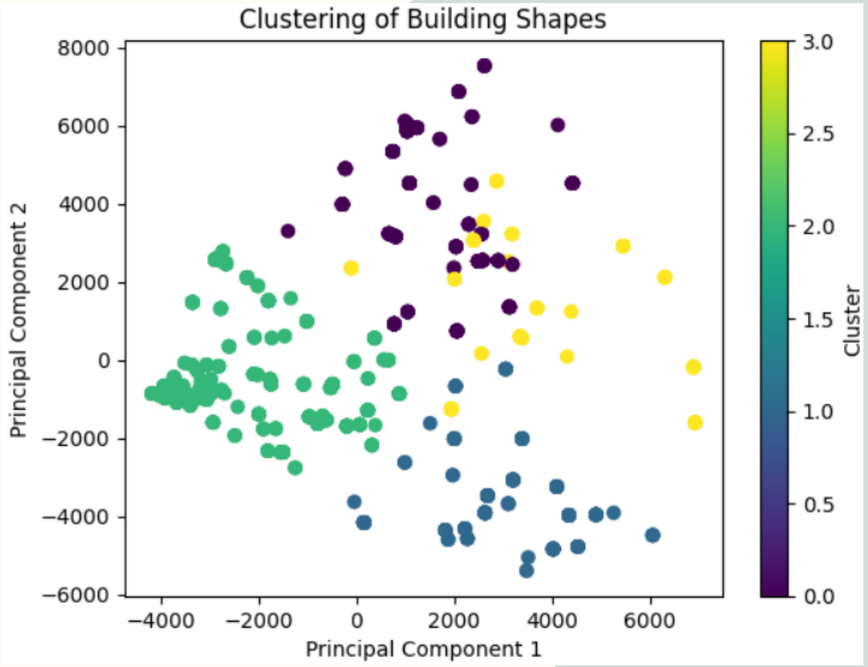
- Use of OpenCV, sklearn, ML algorithms
- Pre-processing images
- Feature extraction and standardization
- Dimensionality reduction using PCA
- Analysing various features and choosing relevant ones as criteria for classification
- Using unsupervised learning to create categories, varying number of clusters and components
- Visualisation and storage of clusters of images
- Minimising Euclidian distance between cluster centroid and input parameters to obtain suitable design family

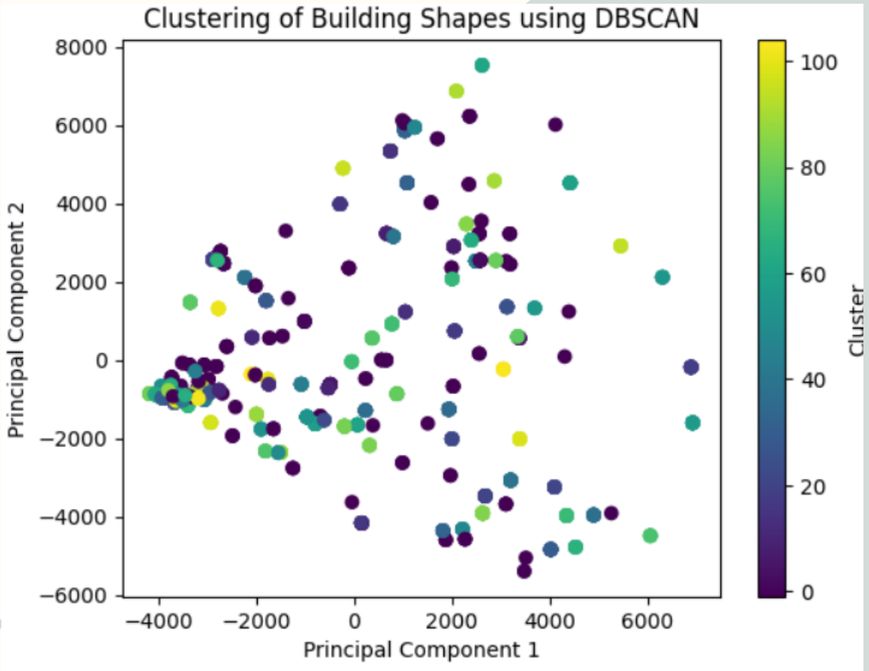
Results summary

To the right are families/design groups/clusters formed based on shape, using the approaches of **k-means** and **DBSCAN** clustering respectively.

Classification based on complexity: 4 criteria used: **symmetry, perimeter to area ratio, convexity defects** and **shape complexity** to obtain 3 clusters containing **628 low** complexity designs, **381 medium** complexity designs and **174 high** complexity designs.

Obtained output of most closest design family for gross input parameters.





Data preparation

Preprocessing:
-Converted the images to grayscale, using cv2 module
-Thresholding to isolate the yellow shapes from the white background, using a binary mask

Feature extraction
Resizing and flattening the image to a 1-D array

Learning outcomes:
Extensive use of cv2 module and functions like contourArea, arcleangth, convexHull, etc.

PCA

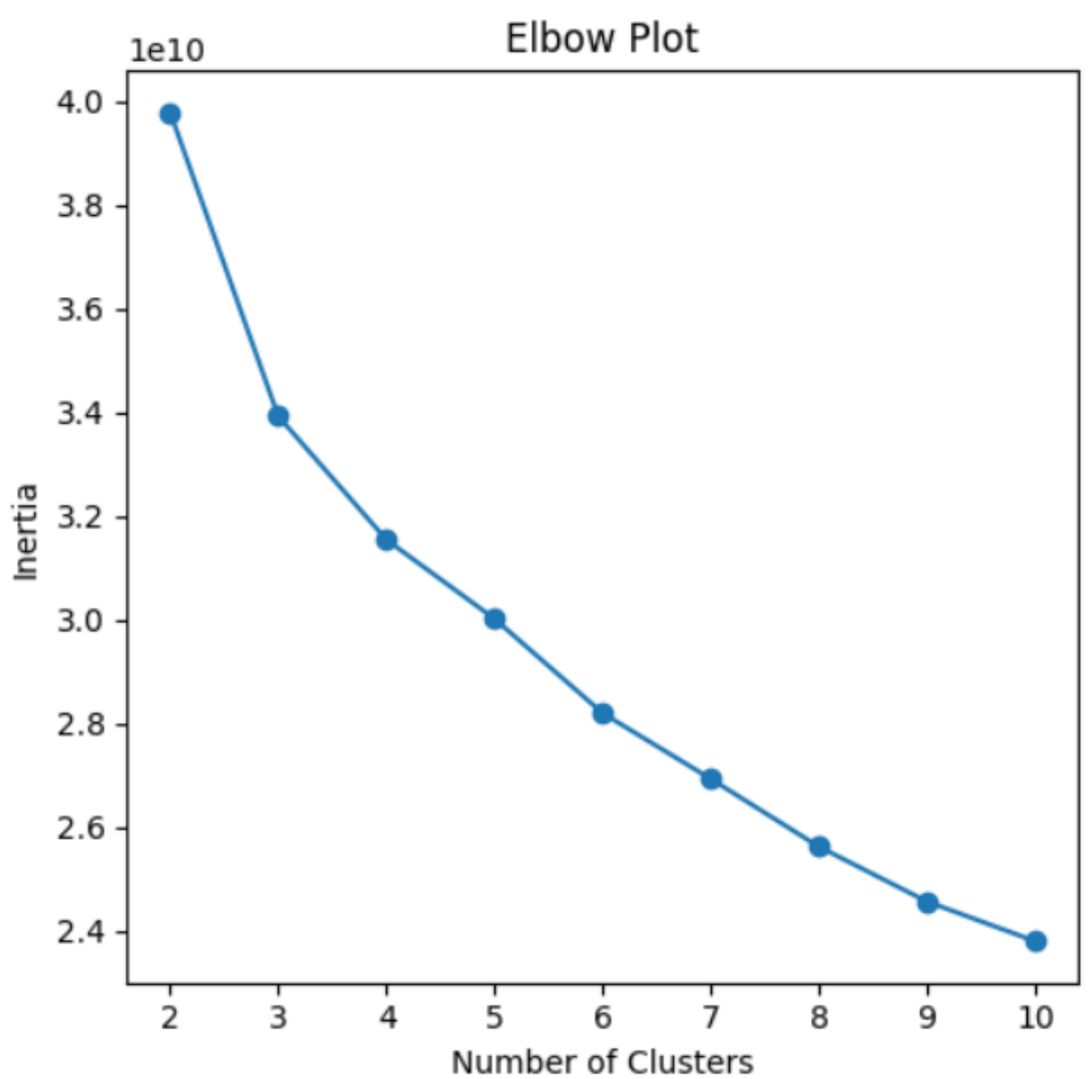
- As a part of processing the images, we applied dimensionality reduction using PCA.
- We want noise reduction and improved cluster separation.
- We want to transform the original features into a smaller number of principal components that capture most of the variance in the data.
- How did we decide the number of components? Say we wanted to retain around 99% variance.
- We accessed cumulative explained variance ratio, using pca.explained_variance_ratio_.
- At n_components=100, we obtained cumulative value of 98.96%. So, we choose n=100.

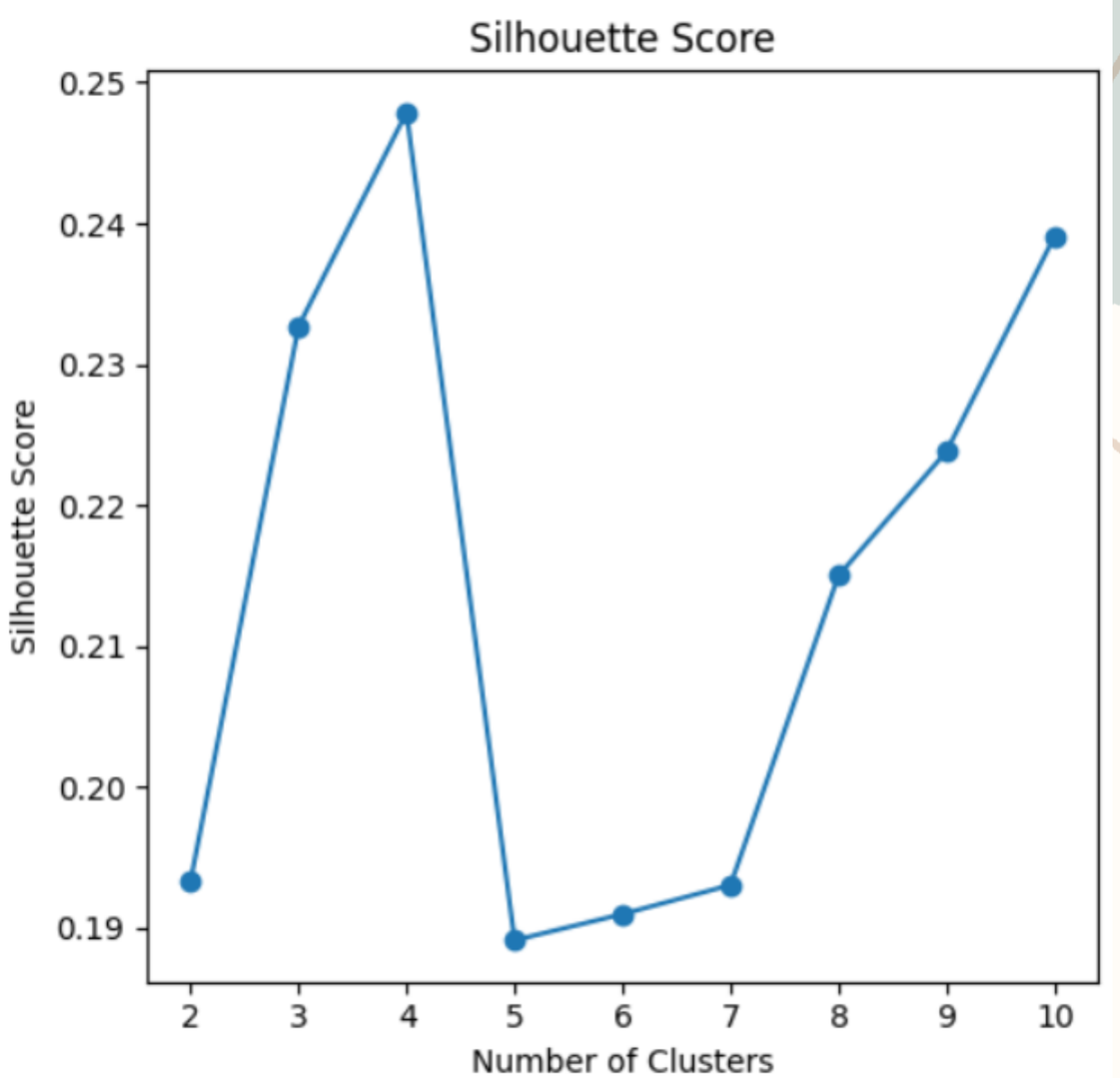
Categorization of designs on the basis of shape

Approach 1: Clustering using K-Means algorithm

Once we have the preprocessed images and have extracted features and reduced dimensionality, we wish to classify them into clusters of various shapes. How do we decide how many clusters to make? We varied number of clusters from 2 to 10 to see the effect on the following:

- Elbow Plot: We want to minimise sum of squares within the cluster, i.e., inertia. We plotted inertia vs no. of clusters, to see the ‘elbow point’ where increasing number of clusters does not decrease inertia significantly.
- Silhouette score: It measures how similar an image is to its own cluster compared to other clusters. We wish to maximise this.

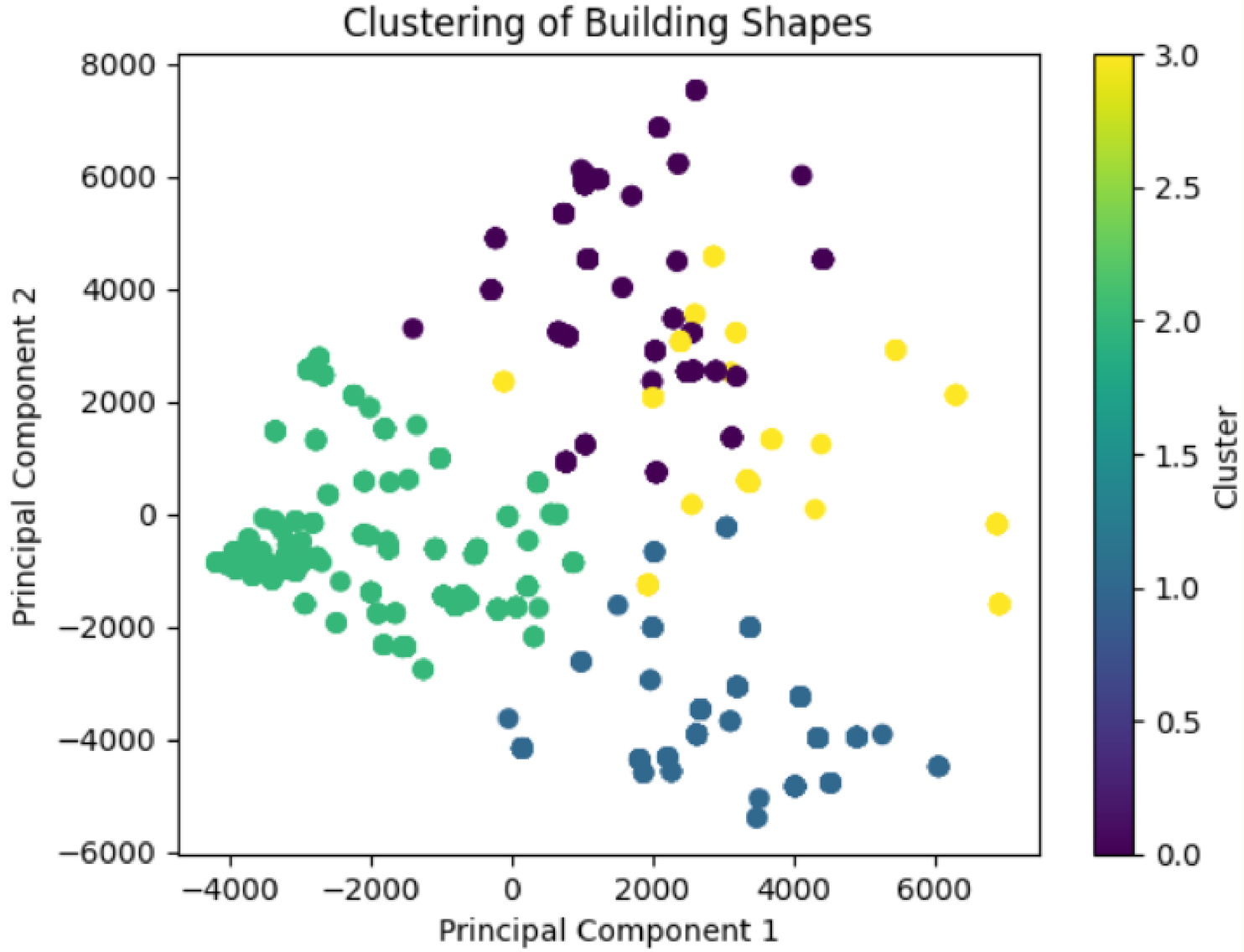




Based on the plots obtained, we deduce that n=4 is optimal, since it maximises silhouette score and decrease in inertia is low after n=4.

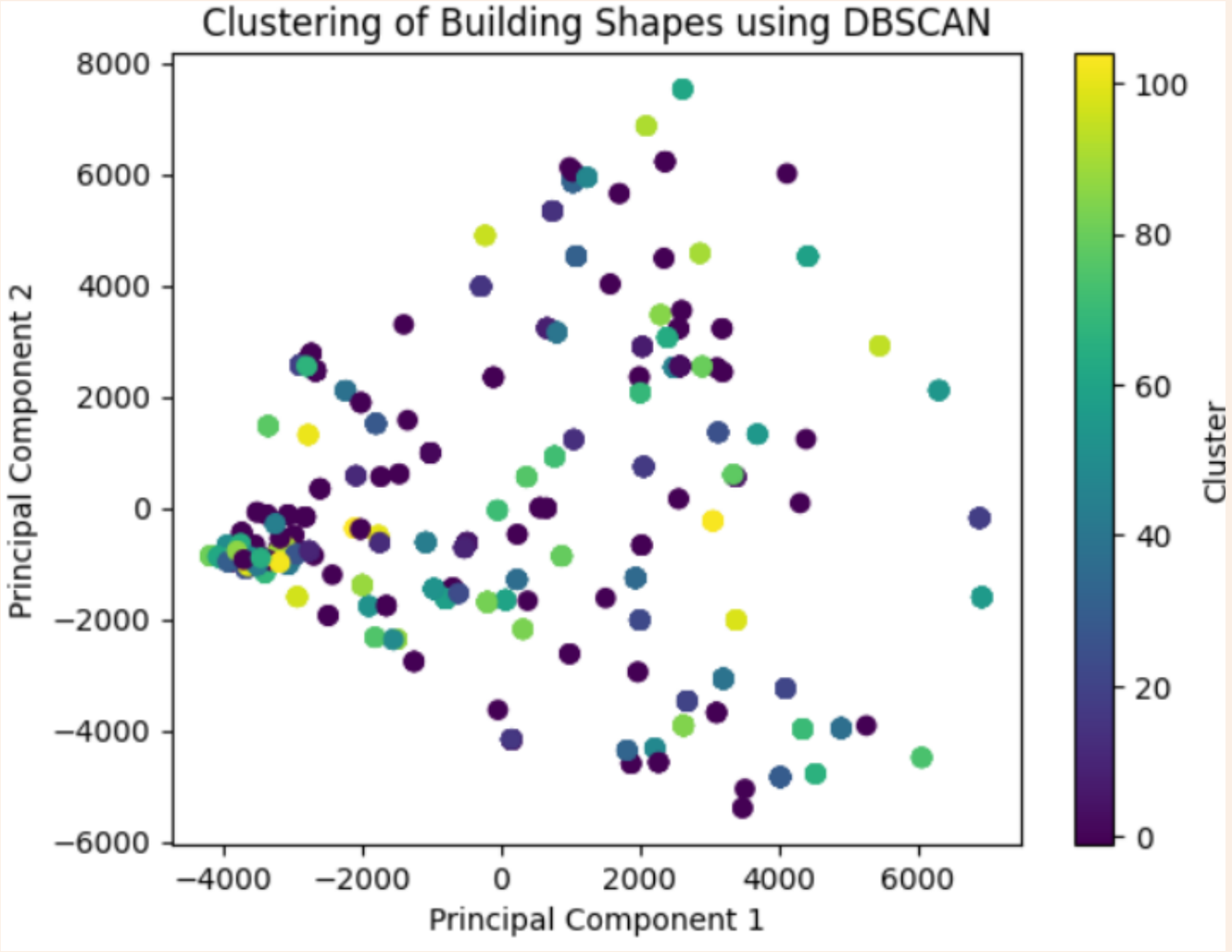
Now, we applied k-means clustering algorithm for 4 clusters and visualised the scatter plot to see the 4 categories of shape that the images have been put into

Green, blue, yellow and violet depict the 4 shape clusters. We then stored these clusters in the form of a dictionary, with 4 key-value pairs and each value being a list of indexes of images in a particular cluster.



Approach 2: Clustering using DBSCAN algorithm

- DBSCAN(Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm, but unlike kmeans it does not require us to specify number of clusters beforehand. It also does not assume clusters to be spherical and similarly-sized.
- It identifies core points and border points.
- It takes parameters:
- Epsilon: maximum distance between 2 points to consider them as neighbouring points.
- Min_samples: number of samples in neighbourhood to consider it a core point.
- These are the clusters obtained using DBSCAN, from sklearn.cluster



Classifying designs as low, medium and high complexity

Steps:

- Analysis and identification of relevant criteria
- Feature extraction
- K-means clustering

Symmetry

It can be calculated by flipping the image to obtain mirror image and calculating mean difference between the 2 images. Lower the difference, higher the symmetry, lower the complexity.

Convexity defects

To detect irregularities or concavities in the yellow shape, using cv2.convexHull and cv2.convexityDefects. More the defects, complexity may be higher.

What defines complexity of a design?

Perimeter to area ratio

It can be calculated using cv2.contourArea and cv2.arcLength. Higher ratio indicates a more irregular shape, which indicates higher complexity.

Shape complexity

It can be calculated by counting the number of contour points using cv2.findContours and len(contour). Irregular boundaries or multiple components may indicate high complexity.

Now that we defined 4 criteria to group the designs based on complexity, we defined a function that calculates these criteria for each design using required functions from cv2 module.

Based on these extracted features, we carried out k-means clustering, for 3 clusters, for the preprocessed designs.

We stored clusters as a dictionary where 628 images are classified as low complexity, 381 as medium and 174 as high. Shown ahead are few examples of designs in each category.

Low Complexity

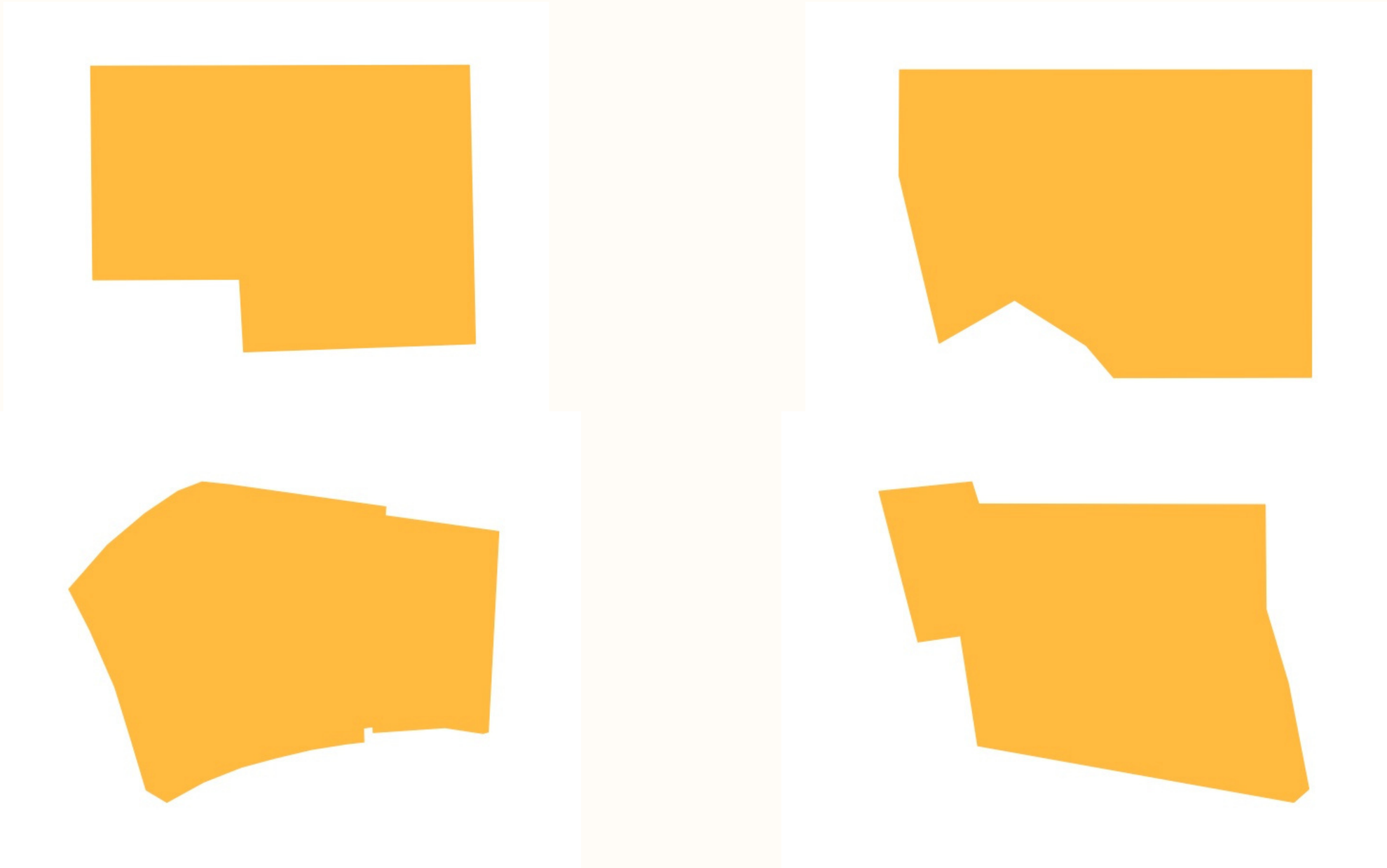








Medium Complexity



High Complexity



Retrieval of relevant prior layouts based on gross parameters

Steps:

- Applying bounding boxes on the images
- Accessing previous dictionaries of complexity categories and design families
- Finding centroid of each design family, for each input parameter and picking the one closest to the input values

1. We used cv2.boundingRect to obtain dimensions of bounding box for each image

2. We created the lists layout_widths layout_heights and layout_areas in Q2, which contain length of bounding box, width of bounding box and layout area of each image

3. We accessed k_clusters_dict, a dictionary containing the 4 clusters of design families created in Q1, and classes, a dictionary containing 3 key-value pairs for low, medium and high complexity created in Q2.

4. For each design family, we chose indexes of those images common to the input complexity class and the design family.

5. Then for each design family, we calculated the mean value of area, length and width, which is the cluster centroid and calculated euclidean distance between cluster centroid and input parameters.

6. Finally, we chose the most appropriate design family as the one with minumum distance.

Sample output

```
# User-specified parameters
complexity = "Low"  # Low, Medium, or High
length = 65  # Dimensions of bounding box
width = 15   # Dimensions of bounding box
area = 1500  # Area of the image
```

```
✓ 0.0s
The closest shape-based cluster for complexity 'Low' and dimensions (length=65, width=15) with area=1500 is '2'
```

Thus, designs belonging to cluster 2, in Q1, are most suitable for this user.

Why trust our solutions?

- Our solution approach involved robust data handling and pre-processing.
- We performed complexity classifications based on 4 parameters, making our criteria comprehensive.
- The visualization and interpretability of the results make it adaptable for other use-cases.
- The detailed analysis performed and results obtained can be scaled to handle growing data volumes.

Thank you!